



**INSTITUTO FEDERAL DO NORTE DE MINAS GERAIS -
CAMPUS MONTES CLAROS**

BACHARELADO EM CIÊNCIA DA COMPUTAÇÃO

RELATÓRIO: TRABALHO PRÁTICO 1A

DISCIPLINA: PROCESSAMENTO DIGITAL DE IMAGENS

ALUNO: Gabriel Davi Mota Baia

PROFESSOR: Wagner Ferreira de Barros

**MONTES CLAROS - MG
2026**

1 Introdução

Este relatório apresenta as soluções desenvolvidas para a Atividade Prática 01 da disciplina de Processamento Digital de Imagens. O foco principal foi a utilização da biblioteca **NumPy** para manipulação de imagens através de vetorização e fatiamento de matrizes, respeitando a restrição de não utilizar laços de repetição (*for/while*) para o processamento de pixels.

2 Desenvolvimento e Resultados

2.1 Questão 1: Fatiamento e Mescla (slicing.py)

Nesta questão, gerei uma máscara geométrica e um degradê horizontal. Utilizei a função `np.where` para realizar a interseção visual.

```
1 #1 - fatiamento (slicing)
2     #mascara
3
4 #Gere uma imagem de 200 x 200 pixels com o fundo totalmente preto (valor
5     0)
6 img_1a = np.zeros((200,200,3), dtype=np.uint8)
7
8 #ex: matriz[50:150, 50:150] e quadrado branco valor = 255
9 img_1a[50:150, 50:150] = [255,255,255]
10
11 #mostrar imagem
12 plt.imshow(img_1a)
13 plt.title("mask")
14 plt.axis("off") #deixa so imagem
15 plt.show() #imprime tela
16
17     #degrade
18
19 #imagem 200 x 200, desta vez contendo um degrade horizontal, indo do
20     lado preto (0) na esquerda ate branco (255) na direita.
21
22 y,x = 200,200
23 degrade = np.linspace(0,255,x,dtype=np.uint8) #0,255 para sair de preto
24     e ir a branco passa x pq horizontal
25
26 img_gray = np.tile(degrade, (y, 1))
27 img_1b = np.stack([img_gray]*3, axis=2)
28
29 plt.imshow(img_1b)
30 plt.title("Degrade")
```

```

29 plt.axis("off") #deixa so imagem
30 plt.show()
31
32
33     #Merging
34
35 #Utilize a funcao np.where(ou multiplicacao de matrizes divididas por
    255) para combinar as duas imagens.
36 # A regra e: onde a Imagem 1a for branca (255), mostre o degrade (Imagem
    1.b). Onde for preta, mantenha o fundo preto (0). Exiba a interseccao
    visual final.
37 img_1c = np.where(img_1a == 255, img_1b, 0).astype(np.uint8)
38
39 plt.imshow(img_1c)
40 plt.title("Mesclagem")
41 plt.axis("off")
42 plt.show()

```

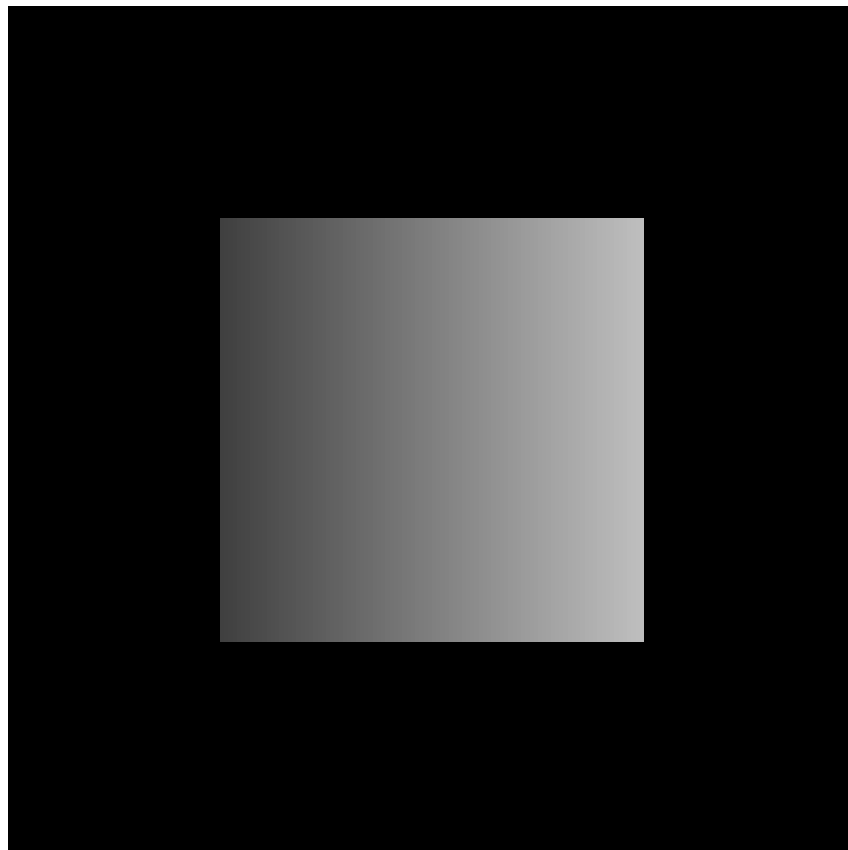


Figura 1: Resultado da mesclagem entre máscara e degradê.

2.2 Questão 2: Espaço RGB (rgb_Space.py)

Foi realizada a manipulação dos canais de cor, incluindo a remoção do canal Alpha, a inversão dos canais Vermelho e Azul, e a criação de um filtro vermelho puro zerando os canais Verde e Azul via fatiamento.

```
1 #2 - Brincando com Canais de Cor(RGB Space)
2     #Leitura
3
4 path = "img/q2_a_original.png"
5
6 img_alfa = plt.imread(path)
7 print("Shape original:", img_alfa.shape)
8 plt.imshow(img_alfa)
9 plt.title("Com Alfa")
10 plt.axis("off")
11 plt.show()
12
13 if img_alfa.shape[-1] == 4: #tirando alfa
14     img_NoAlfa = img_alfa[:, :, :3] #tira do alfa
15 else:
16     img_NoAlfa = img_alfa
17
18 print("Shape final (RGB):", img_NoAlfa.shape)
19
20 plt.imshow(img_NoAlfa)
21 plt.title("Sem Alfa")
22 plt.axis("off")
23 plt.show()
24
25
26     #Invert RGB com fatiamento
27
28 img_invert = img_NoAlfa.copy()#copia imagem
29
30 img_invert[:, :, [0, 2]] = img_invert[:, :, [2, 0]] #troca o vermelho pelo
    azul se fosse trocar vende teriamos que pegar o valor 1
31
32 plt.imshow(img_invert)
33 plt.title("R e B Invert (fatiamento)")
34 plt.axis("off")
35 plt.show()
36
37
38     #Big Red copia foto original(com alfa)
39
40 img_red_filter = img_NoAlfa.copy()
```

```
41 img_red_filter[:, :, [1,2]] = 0 #zera verde e azul
42
43 plt.imshow(img_red_filter)
44 plt.title("Filter Red")
45 plt.axis("off")
46 plt.show()
```

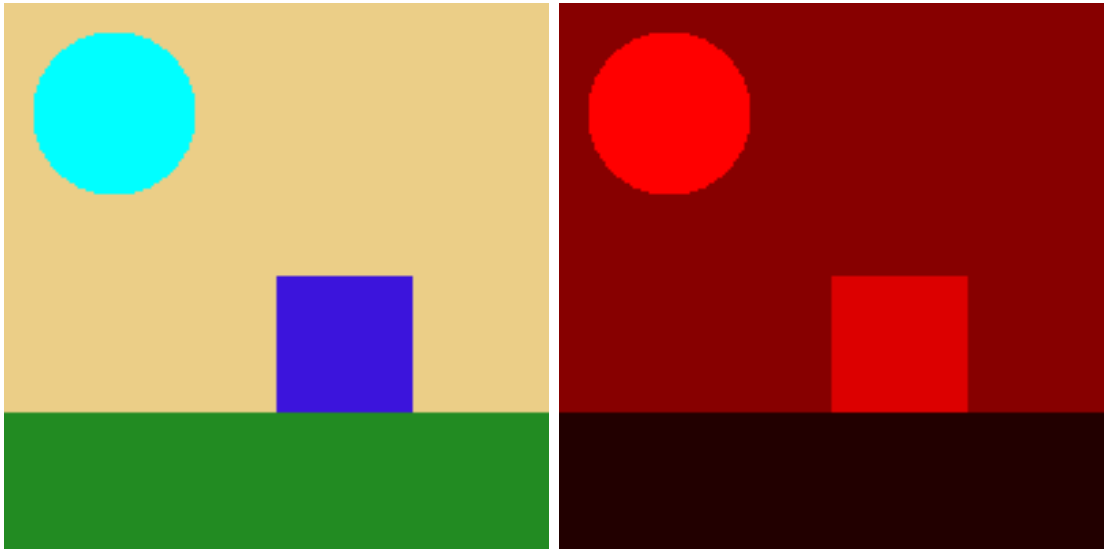


Figura 2: Esquerda: Inversão R-B. Direita: Filtro Vermelho.

2.3 Questão 3: Estatística e Limiarização (statistics_threshold.py)

A imagem foi convertida para tons de cinza e submetida a uma limiarização automática baseada na média global de intensidade dos pixels.

```
1 #3 - calculos estatisticos e limiarizacao (statistics)
2     #Media e Gray
3
4 path = "img/q2_a_original.png"
5 img = plt.imread(path)
6
7     # Media dos canais e converte para cinza
8 media_canais = np.mean(img, axis=2)
9 img_gray_uint8 = (media_canais * 255).astype(np.uint8)
10
11 plt.imshow(img_gray_uint8, cmap='gray')
12 plt.title("Imagem em tons de cinza")
13 plt.axis("off")
14 plt.show()
15
16     #Media e plotar histograma
17 media_total = np.mean(img_gray_uint8)
18 print("M dia total:", media_total)
19
20 plt.figure(figsize=(8,5))
21 plt.hist(img_gray_uint8.flatten(), bins=255, range=(0,255), color='gray',
22         )
23 plt.title(f"Histograma da imagem em tons de cinza: {media_total:.2f}")
24 plt.xlabel("Intensidade")
25 plt.ylabel("N mero de pixels")
26 plt.show()
27
28
29     #Limiarizacao Analitica Automatica:
30
31 # Criar m scara bin ria usando a m dia como threshold
32 mask = np.zeros_like(img_gray_uint8, dtype=np.uint8)
33
34 # Define pixels acima ou igual m dia como 255 (branco)
35 mask[img_gray_uint8 >= media_total] = 255
36
37 plt.imshow(mask, cmap='gray')
38 plt.title(f"Limiariza o autom tica (Threshold = {media_total:.2f})")
39 plt.axis('off')
40 plt.show()
```



Figura 3: Histograma de frequências e média de intensidade.

2.4 Questão 4: Chroma Key (chroma_key.py)

Implementei a técnica de Chroma Key isolando o azul do céu. Criamos um novo fundo noturno com degradê vertical e aplicamos a mesclagem booleana (Figura 4).

```

1 #4 - Chroma Key
2     #isolamento
3
4 path = "img/q2_a_original.png"
5 img = plt.imread(path)
6
7 if img.shape[-1] == 4: #tirando o alfa
8     img = img[:, :, :3]
9
10 ceu_rgb = np.array([135, 206, 235]) / 255.0
11 tolerancia = 0.05 # 5% de tolerancia no valor acima, quanto maior mais
    pixels longe do valor atual de rgb informado ele vai pegar
12
13 mask_ceu = (np.abs(img[:, :, 0] - ceu_rgb[0]) <= tolerancia) & (np.abs(img
   [:, :, 1] - ceu_rgb[1]) <= tolerancia) & (np.abs(img[:, :, 2] - ceu_rgb
    [2]) <= tolerancia)
14
15 plt.imshow(mask_ceu, cmap='gray')
16 plt.title("Mask do C u Azul")
17 plt.axis("off")
18 plt.show()
19
20
21     #Novo fundo
22
23 y, x, canais = img.shape

```

```

24
25 new_fundo = np.zeros(img.shape, dtype=img.dtype) # Cria a tela preta
26
27 coluna_degrade = np.linspace(0.0, 0.6, y).reshape(-1, 1) #come a no
    preto (0.0) e vai at o Azul Escuro (0.6) na altura (y)
28 matriz_degrade = np.tile(coluna_degrade, (1,x))
29 new_fundo[:, :, 2] = matriz_degrade #aplica degrade somente no rgb azul
30
31 plt.imshow(new_fundo)
32 plt.title("Novo Fundo (Azul Escuro para Preto)")
33 plt.axis("off")
34 plt.show()
35
36
37     #Aplica Chroma key
38
39 img_chroma = img.copy() #copia imagem
40 img_chroma [mask_ceu] = new_fundo[mask_ceu] #pega os pixels do ceu
    separado na quest o A e os pixels do ceu da imagem gerada na
    quest o B
41
42 plt.imshow(img_chroma)
43 plt.title("Imagem Com Chroma Key")
44 plt.axis("off")
45 plt.show()

```

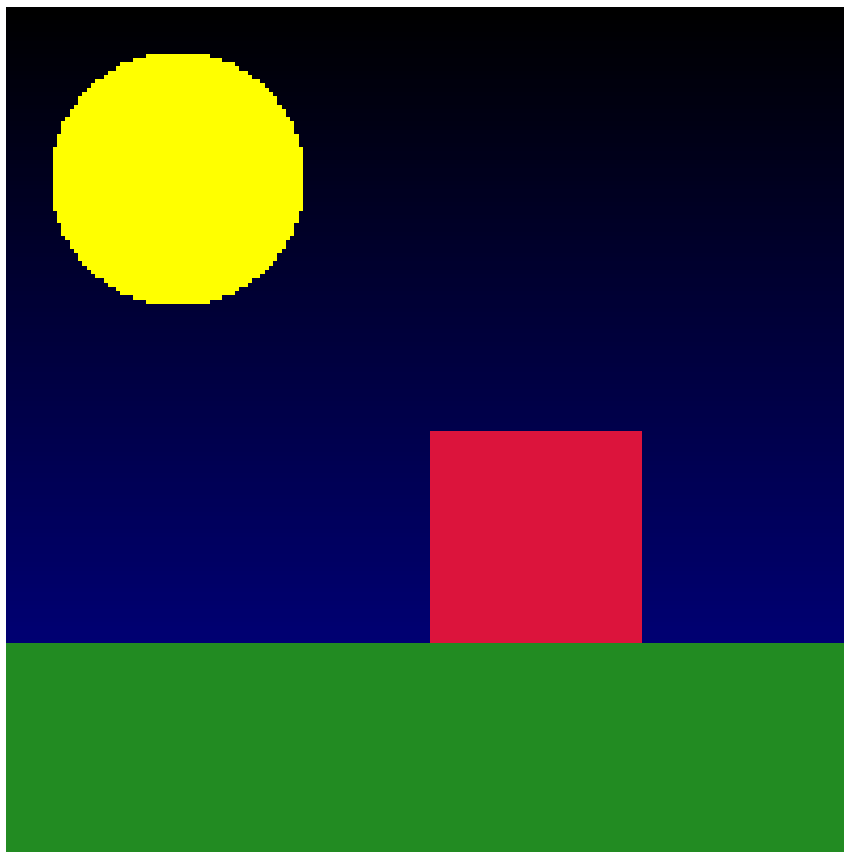



Figura 4: Resultado final do efeito Chroma Key aplicado.

3 Conclusão

A atividade permitiu consolidar o conhecimento sobre manipulação de tensores e álgebra linear aplicada a imagens. A vetorização do NumPy provou ser uma ferramenta poderosa, resultando em códigos performáticos e de fácil manutenção.

4 Referências de Código (GitHub)

<https://github.com/GabrielDavi7/PDI/tree/main/Atividade-01-A>